

QOMM

query-oblivious
market making

数値は全て実測

予測を先に書き、
外れた分も残してある

注文を見せずに 最良気配を出す

秘密計算による RFQ
その署名・証明・決済・制度上の位置

対象: 関連技術に詳しい技術者、および金融実務者 | 2026 年 8 月

3行で

1. いまの RFQ は、**値段を聞くために注文内容を教える**方式である。16 社に聞けば、**成立しなかった 15 社**も「誰が・何を・どちら向きに・いくつ・いつ」を持ち帰る。
2. 提案する方式では、注文を 7 つの計算ノードに**断片として**配り、MM が事前に預けた**付けルール**を秘密のまま適用する。返るのは**価格と、その価格が本当に 16 社中の最小だったという公開検証可能な証明**。
3. 実装は研究プロトタイプとして動いており、費用は測ってある。残っているのは暗号ではなく、**鍵管理・選択的開示・制度上の位置**である。

0.617 s

同一メトリックで 1 クォート (N=7,
M=16)

AUC 0.500

成立しなかった注文の秘匿 (当て推
量と同じ)

700 /秒

MM が見立てを更新できる頻度 (16
社時)

出所・注: 全て本資料の後半で出所と条件を示す。

この資料の読み方

本資料は**二つの読者**を想定している。同じ主張を、二通りの言葉で置いた箇所には次の枠が付く。

実務者の読み方

市場・執行・規制の言葉。「これは誰の costs をどう変えるか」

技術者の読み方

構成・費用・仮定の言葉。「何を仮定し、何を証明し、何を測ったか」

- 色には意味を持たせてある。**琥珀は漏れる情報**、**青緑は漏れない情報**。最後まで一貫している。
- 数値は太字が実測。推定と実測は必ず書き分ける。
- **予測を外した箇所も残してある** (§5 末尾に一覧)。当てた予測だけを見せるのは測定の作法として誠実でない。

出所・注: 本資料の数値は全て公開リポジトリの成果物 (JSON) と、それを生成するコマンドに対応する。

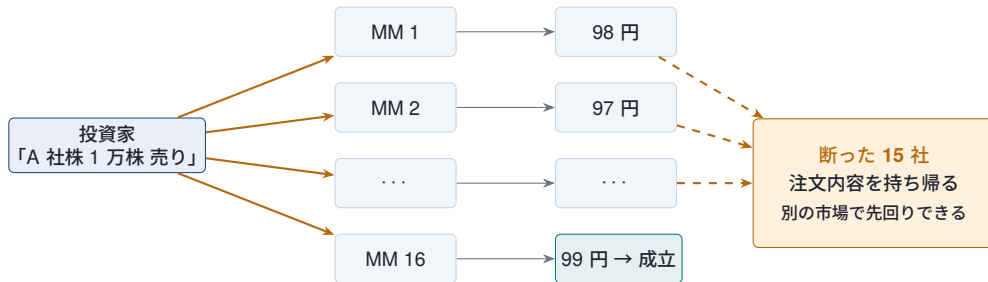
§1

いまの RFQ は何を漏らしているか

値段を聞くには、注文を教えなければならない

具体例: ある投資家が1万株売りたい

銘柄・売り・1万株・時刻



実務者の読み方

約定しなかった相手が板情報より価値のある情報を無償で得る。大口・低流動性ほど、この情報の値段は高い。

技術者の読み方

リークは実装の不備ではない。「問い合わせ＝開示」がプロトコルの定義そのもの。

これは実装の不備ではなく、方式の性質である

なぜ債券は板取引でないのか

- 銘柄数が桁違い（社債は数万～数十万）。板を立てるとほぼ全て空
- 1 件が大きく、頻度が低い。板に出した瞬間に意図が読まれる
- 在庫リスクをディーラーが負う必要がある
- 情報非対称が大きく、**相手を選ぶ**必要がある

→ だから**相対で聞く (RFQ)**。板が使えないから RFQ なのであって、RFQ が劣った選択肢なのではない。

そして RFQ の定義上、こうなる

値段を知るには聞くしかない。
聞けば、聞かれた側は注文を知る。

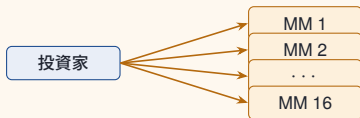
16 社に聞けば 16 社が知る。
成立するのは 1 社なので、
15 社は対価なしに情報を得る。

これを「聞く相手を減らす」で解こうとすると、**価格競争が落ちる**。執行コストの二つの成分が、正面から衝突している。

出所・注: 第 5 回講座資料（金融資産市場の仕組み）§3.2、CGFS 2016、IOSCO PD700 (2022)。

同じ問いを、誰にも聞かずに立てる

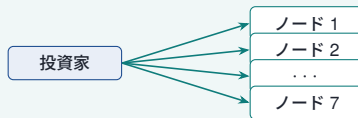
いま 注文そのものが、16 社に届く — 銘柄・向き・数量



値段（成立する 1 社から）

注文を知る者 **16** 社
断った 15 社は、対価なしに持ち帰る

QOMM 出ていくのは持ち分だけ — 1 つ見ても乱数



覆われた鍵（全員が同じ数字を見る）

注文を知る者 **0** 社
16 社の方針も、ノードには破片でしか届かない

	RFQ 片側	RFM 両側	RFS 連続配信
何を聞く いま漏れる QOMM	この向きで、いくら 銘柄・向き・数量 何も	売値と買値の両方 銘柄・数量（向きは隠れる） 何も	一定間隔で気配を流し続ける 意図の時系列。最も重い 何も。在庫は連鎖するが秘匿の まま

三つで漏れが同じなのは偶然ではない — 回路は同じで、違うのは何本の値段を出すかだけ。

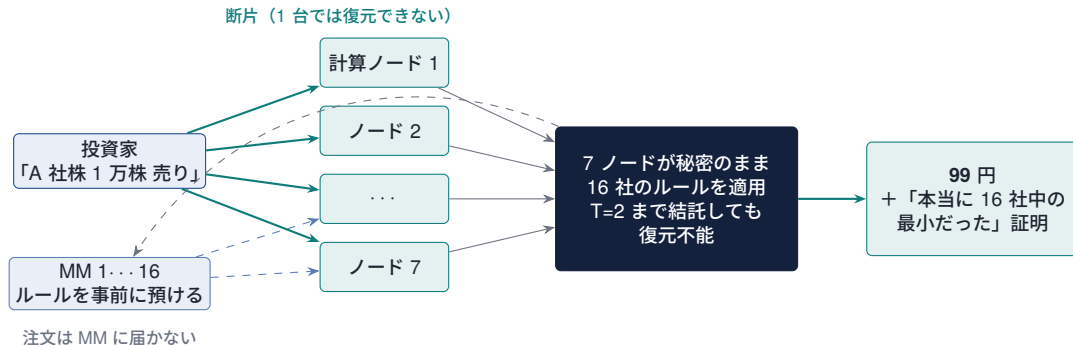
出所・注: モード名は実装のもの (gen_qomm.py --mode)。RFM は向きを回路に与えないので暗号なしでも向きは隠れる — その分を MPC の効果として数えてはいない。

§2

提案する形

注文は誰にも届かず、値段だけが返る

同じ例を、提案方式で



断った 15 社は、何も持ち帰らない。 勝った 1 社は必ず知る (決済するので原理的に不可避)。

役割を分ける ― 分散する側と計算する側は別主体

ここを混ぜると設計が壊れる。外部レビューで最初に指摘された点でもある。

役割	持っている秘密	することしないこと	実装
トレーダー	注文内容、認証鍵、答えを受け取るマスク	断片を配る。計算には参加しない	Trader
マーケットメイカー 計算ノード (7)	値付けルール、在庫 鍵シェア、署名シェア	断片を配る。計算には参加しない 計算する。値そのものは見えない	MarketMaker Computing

実務者の読み方

注文を出す側と値段を出す側は、システムの運営者ではない。運営者（ノード）が単独で顧客の注文も業者の値付けも読めない構造。

技術者の読み方

入力当事者は `get_input_from` で全ノードに加法分散し、回路が総和を取る。1 ノードが 1 社の方針を平文で持つ旧構成を、この分離で潰した。

出所・注: `qomm_transport/roles.py`。分散の総和は整数上で取るので法の一致が要らない。統計的秘匿は 2^{-40} 。

誰が何を知るか ― この表が主張の全体

	負けた MM	勝った MM	計算ノード (単独～2 社)	テ
注文の銘柄・数量・方向	知らない	知る	知らない	自
注文を出したのが誰か	知らない	会場経由	知らない	自
注文があったこと自体	知りうる	知る	知る	自
他社の値付けルール	知らない	知らない	知らない	知
最良価格	知らない	自分の値	知らない	知
それが最小だった証明	—	—	—	検
参照価格（市場ミッド）	公開	公開	公開	公

正直に言うべき 1 行がある。方針は評価ごとの秘密入力なので、MM は「何かが起きた」ことは知りうる。スロット単位にまとめれば「スロットが動いた」まで落ちるが、そこは未検証。中身が漏れないことと、発生が漏れないことは別の主張である。

出所・注: 秘匿の定量評価は `sim_matrix.json` (成立しなかった注文の識別 AUC 0.500 = 当て推量と同じ)。

§3

プロトコル署名

回路が何を計算するかを、1枚の宣言で固定する

値付けルールの宣言を「プロトコル署名」として出す

秘密計算の説明は、たいてい「何が隠れるか」に寄る。しかし実務で先に要るのは何が表現できるのかである。MM は「この仕組みで自分の商売ができるのか」を、暗号を読まずに判断したい。

そこで、DSL のルール宣言そのものを公開仕様として出す。パラメータ・その範囲・状態・入力・価格式・適格条件が 1 ファイルに全部ある。これが両者の接点になる: 実務者は「自分の建て方が書けるか」を読み、技術者は「何が回路になるか」を読む。

- 曖昧さが消える。「柔軟に値付けできます」ではなく、書ける式が確定する
- 監査に直結する。宣言はダイジェストで固定され、登録・改竄検知に使われる
- 規制対応の材料になる。最良執行方針に「何を最適化しているか」を正確に書ける
- 限界が先に見える。書けないものが読めば分かる（これが一番重要）

出所・注: `qomm_dsl/language.py` (構文と検査)、`qomm_dsl/registry.py` (ダイジェストと登録)。

署名の全文 — 参照価格を使う市場

```
param ask_level[-2000,2000], spread[2,400], slope[0,16], invcoef[0,8], maxqty[1,1000]
param expiry[0,1000000], active[0,1], use_ref[1,1]
state inv[-4000,4000]
input qty[1,400], ref_mid[90000,110000], now[0,1000000]

ask      = use_ref * ref_mid + ask_level + slope * qty + invcoef * inv
bid      = use_ref * ref_mid + ask_level - spread - slope * qty + invcoef * inv
eligible = (qty <= maxqty) and (expiry > now) and (active == 1)
```

実務者の読み方

param は自分が決める値。**state** は約定で動く在庫。
input は取引ごとに来る値。

読み方は「市場のミッドから何ティック離して、どれだけの幅で、サイズと在庫でどうずらすか」。ディーラーの在庫スキュー型そのもの。

技術者の読み方

param と **state** は秘密入力、**input** も秘密。**ref_mid** だけは公開表からの秘匿参照（どの行かが秘密）。

全体が SIMD 2 乗算+比較の深さ 1 層に落ち、勝者選択は深さ $\log_2 M$ のトーナメント。

どの項も「使わない」を選ぶ

項	消し方	意味
サイズ調整	<code>slope = 0</code>	サイズに依らない一律の値段
在庫調整	<code>invcoef = 0</code>	在庫を値段に反映しない
その MM 自体	<code>active = 0</code>	この銘柄では建てない
数量上限・期限	上限最大／期限を遠く	実質無制限
スプレッド	<code>half = 1</code>	下限が 1 ティック（完全な 0 は不可）
参照価格	<code>use_ref = 0</code>	後から足した。以前は選べなかった

実務者の読み方

社債には連続的なミッドが存在しない。存在しない指標からのオフセットで建てろというのは、市場の実態に合わない。

技術者の読み方

`anchored = mid + use_ref * ref`。既に乗算 2 つある深さ 1 の層に 1 つ増えるだけなので深さが動かない。

実測（同一マシン・同一セッション、両方とも検証通過）：ラウンド 64 → 64（不変）、通信 +0.26%、実時間は誤差の内。

出所・注: `use_ref_cost.json`。予測は「ラウンド不変・通信+1〜3%」で、通信は 5 倍の過大見積もりだった。

2つの市場、2つの署名

参照相対 `use_ref = 1`

`ask_level[-2000, 2000]` ($\pm 2\%$)

市場水準は公開の参照価格が運ぶ。MM は自分の差分だけ持つ。

向く市場: **FX**・金利・上場物・暗号資産の主要ペア

→ §4 の不変性が効き、計算の遅れが無料になる

絶対値 `use_ref = 0`

`ask_level[40000, 200000]`

参照価格を一切足さない。MM が水準そのものを持つ。下限は下方調整の最大 **38800** を超えるように置く — でないと負の価格を出す登録が通る。

向く市場: 社債・仕組物・気配の薄いもの

→ 不変性は効かない。ただし元々動きが遅いので害が小さい

範囲を広げる代償は**範囲証明の幅**である。12 ビットのパラメータは 16 ビットの証明に丸められ、18 ビットなら 32 ビットになる。それが選択の全コストで、**会場が全員分を決めるのではなく MM が市場ごとを選ぶ**。

出所・注: `qomm_dsl/examples/quote.rule` と `quote_absolute.rule`。範囲証明は Bulletproofs で 2 の冪に丸められる。

§4

不変性

参照価格が動いても、勝者は変わらない

不変性とは何か ― 一文で

参照価格は全 MM の気配に同じだけ足されるので、参照価格が動いても順位は変わらない。

全員に同じ高さの下駄を履かせているだけなので、下駄の高さが変わっても背の順は変わらない。そして価格の方は、参照価格の差分を足すだけで直る。

	開始時 (参照 100.00)	24 秒後 (参照 100.02)
MM 1 (+0.03)	100.03	100.05
MM 2 (+0.05)	100.05	100.07
MM 7 (+0.01)	100.01 ← 最安	100.03 ← やはり最安
MM 12 (+0.02)	100.02	100.04

全員が同じ 0.02 だけ動いた。勝者 **MM 7** は最初から正しく、価格は $100.01 + 0.02 = 100.03$ と足し算 1 回で直る。

なぜ順位が変わらないと言い切れるか

参照価格が入る箇所は回路に **1 つだけ**で、そこは全 MM 共通である。

```
anchored = mid + use_ref * ref
ask       = anchored + half + depth + skew
```

← 全 MM に同じ値
← ここから先は MM 固有

そして**適格性ゲート**（銘柄一致・数量上限・有効期限・稼働中）は参照価格を一切読んでいない。したがって参照価格 r に対して $\text{cost}_i = (1 - 2d)r + (\text{MM } i \text{ 固有の項})$ という形になり、方向 d は 1 件の注文で 1 つなので、 r は全 i に同じ係数で入る。

実務者の読み方

最良執行の観点では「誰を選ぶか」と「いくらで約定するか」が分離されている、ということ。参照価格の出し手は価格水準は動かせるが、業者の選択には介入できない。しかも参照価格は公開なので突合できる。

出所・注: tests/test_reference_invariance.py (33 件)。

技術者の読み方

売買両方向 × 5 つの方針データ × 7 通りの参照価格の移動で確認済み。平文モデル上の性質なので、回路側の番兵値の境界（既定構成で参照価格の約 336 倍）も別途テストしてある。

不変性で何を買えるか — 計算の遅れが無料になる

クロスリージョンの委員会は 1 クォートに 24 秒かかる (§5)。素朴には「24 秒前の参照価格で計算した答えは古い」と思える。そうならない。

1. 計算開始時点の参照価格で走らせる
2. 出てきた価格を、その間に参照価格が動いた分だけ後から補正する
3. 勝者は補正不要。最初から間違っていない

補正はラウンドを消費しない。秘密の値 $\times$ 公開の値は、各ノードが自分の持ち分に掛けるだけで済み、ノード間の通信が要らない。複数銘柄でも、回路が既に持っている「どの銘柄か」の秘密ベクトルに公開の値動き表を掛けて足せば、銘柄を明かさずに正しい補正量だけが出る。単一銘柄なら公開の足し算で、テイカーが自分で行える。

ただしこれは設計が持つ性質で、実装はまだ使っていない。参照価格表は回路のパラメータ設定時に固定され、絶対価格が開示される。直すのは「最後に何を出すか」であって、回路のコストではない。

直らないものが 2 つ: 在庫項（ただし約定時にしか動かないのでミッドよりはるかに遅い）と、有効期限の判定（開始時刻で見ているので、その窓で失効する方針を誤って通す。「開始時刻 + 想定遅延」で判定すれば直り、コストはゼロ）。

どこで壊れるか — use_ref の混在

不変性は「参照価格が全員を同じだけずらす」ことに乗っている。一部の MM だけが参照価格を使うと、一部だけがずれる。

参照価格	A (参照相対 $r + 30$)	B (絶対値 100,020)	勝者
99,980	100,010	100,020	A
100,000	100,030	100,020	B
100,040	100,070	100,020	B

99,990 を跨いだところで入れ替わる。

どこに置くか	誰が決めるか	コスト	遅延補正
銘柄ごと (参照表のその行を 0 にする)	会場	ゼロ	効く
MM ごと (use_ref)	MM	+0.26%	混在すると効かない

両方欲しければ道はある: MM に宣言させたいので、銘柄内で揃っていることを登録要件にする。回路ではなく参加要件の話になる。

出所・注: 現在の実装は MM ごと。銘柄ごとの版は「使える指標があるか」が市場の性質であることに対応し、既存の秘匿表引きに畳み込める。

§5

いくら掛かるか

予測を先に書き、外れた分も残してある

測定の作法

守っていること

- 測る前に理論で予測を書く。当たっても外れても残す
- 比較する 2 本は同一マシン・同一セッションで取る
- 計算結果が正しいことを検証してから時間を採る。間違った計算の時間は時間ではない
- 実測と推定を書き分ける
- 新しい計測器は既知の結果と照合してから信用する

この作法で実際に捕まえたもの

- 差分プライバシーの標本器が理論値より 16% 狭かった（閉形式との照合で発覚）
- 配置計測が 4 条件を測った後、最終行で結果を捨てていた（古いキー参照）
- 改竄検知テストの差し替え文字列が空振りし、無改変のルールを検証して通っていた
- コミットメント検査が短絡して、有り得ない速さを報告していた

測定器そのものが壊れる。だから測定器を既知の結果に当てる工程を省かない。

最初の決定 ― ノードをどこに置くか

実測（M=16、31 ビット、malicious Shamir、N=7、T=2）：

ノード配置	片道遅延	1 クォート	RFS 更新率
同一ラック	0 ms	0.166 s	12.0/s
同一メトロ	1 ms	0.617 s	3.0/s
国内広域	5 ms	1.619 s	1.2/s
東京 – シンガポール	15 ms	3.876 s	0.48/s

15 ms の 3.68 秒のうち **2.13 秒** は純粋な往復時間で、暗号をどう変えても縮まない。**7 ノードを 1 メトロに置くことは、見つけたどの暗号的改良よりも 6 倍効く。**

実務者の読み方

速さと結託の相関のトレードオフ。同一メトロは速いが、7 社が同じ法域・同じ商圈に並ぶと、T=2 の前提は組織的独立性だけに乗る。これは技術ではなくガバナンスの選択。

技術者の読み方

ラウンド数は 70 で一定（銘柄数にも MM 数にも依らない）。したがって実時間は $70 \times \text{RTT}$ の純粋な待ちを、最も遅いリンクの分だけ抱える。

出所・注: rounds.json / placement.json。全条件で平文の答えと照合済み。

地理的独立性は委員会の中では買えない

自然な折衷案は「6 台を 1 メトロ、1 台だけ別の法域」だろう。これは折衷にならない。片道 120 ms（東京 - チューリッヒ級）で実測:

配置	1 クォート	同一メトロ比
全台が近い (1 ms)	0.517 s	—
6 台が近く、1 台だけ 120 ms	22.998 ± 0.733 s	44 倍
6 台が 120 ms、1 台だけ近い	25.682 ± 1.090 s	50 倍
全台が 120 ms	26.148 ± 1.209 s	51 倍

1 台だけ遠いノードが、全台移すコストの 86%を払わせる (15 ms では 82%)。ペナルティは距離とともに縮まず、拡大する。

外した予測。 $70 \times 0.240 \text{ s} = 16.8 \text{ 秒}$ + 計算 1.5 秒で約 18 秒と予測した。このモデルは 15 ms で 1.4%以内に検証済みだったが、実測は 23.0 秒。検証した距離の 8 倍で 28%甘くなるので、さらに外挿してはいけない。

出所・注: placement_intercontinental.json。全条件で平文の答えと照合済み。

24 秒は高いのか ― 実データで価格を付ける

決めるのは遅延の長さではなく、その間に価格が動いたかである。UniswapX の約定は「そのブロックで実際に約定可能だったレート」を持つので、同じ市場が 1 ブロック内部で見せる散らばり（スプレッド・サイズインパクト・手数料ティア＝価格がそもそも 1 つの数字でない度合い）と比べられる。

間隔	ドリフト	ブロック内の床との比
1 ブロック内部	4.8–7.5 bp	—
2 ブロック（～24 秒＝クロスリージョン 1 クォート）	3.6–8.6 bp	1.01 倍（中央値）
8 ブロック（～96 秒）	7.4–9.4 bp	1.34–1.75 倍
25 ブロック（～300 秒）	12.2–19.2 bp	1.63–4.00 倍

24 秒のドリフトは、市場がもともと 1 ブロック内部に持っている価格の不確実性と同じ大きさ。

24 秒かけたクォートは、瞬時のクォートと区別がつかない ― ただし約定した注文の中では。古くなって約定しなかった注文は Fill ログを残さないなので、この標本には入らない。つまり測っている当のものの下流で標本が選ばれており、数字は下界であって、傾く向きは設計に有利な側。

出所・注: staleness.json、上位 4 ペア。ペア数の事後選択は効いていない (2 ペア 1.01 / 8 ペア 1.01 / 16 ペア 0.90 / 32 ペア 1.03)。留

そこから出る配置の結論

- 法域を跨いだ委員会は **RFQ** に使える。24 秒で、メトロ配置では持てない地理的独立性が付く。そして注文漏洩の被害が最大の銘柄（スプレッドが広く、板が薄く、サイズが大きい）が、**24 秒のコスト**が最小の銘柄でもある。経済と物理が同じ方向を指している。
- ストリーム（**RFS**）は **RFQ** より強い。24 秒更新のストリームで、テイカーは好きなときに待たずに執行でき、値段は平均 13 秒前。24 秒の RFQ は **24 秒**待って 24 秒前の値段を得る。遅いストリームの損は**メイカー側**に出る——古い気配は速い情報源を持つ相手に有利な瞬間だけ取られるので、ドリフト分だけ広げて自衛することになる（同じ 4.8–8.6 bp）。
- リージョンを跨ぐ決済はそもそもラウンド律速ではない。共通状態のない 2 台帳はアダプタ署名と期限で決済し、暗号側は **0.06 ms**、露出は両台帳のファイナリティが決める。
- 域外の **MM** は、両方に跨る委員会を必要としない。方針の登録とクォート計算は元々分離しているので、ルールが一度だけオフラインで越境すればよい。

基準は drift(遅延) とその銘柄自身の価格不確実性の比であって、遅延そのものではない。階層はアーキテクチャの決定ではなく銘柄ごとの選択になる。

MMは値付けをリアルタイムで更新できるか (1/2 実測)

方針は評価ごとの秘密入力であって固定登録ではない。MMは新しいシェアを配るだけで考えを変えられ、その経路は **MPC** を一切通らない — クォート計算の遅延とは独立である。

更新範囲	認証	MM 側	ノード側 (1 コア)	16 社時の実効	バイト
全 9 項目	分散のみ	12,503/s	2,371,617/s	12,468/s	6,804
全 9 項目	署名	422/s	1,247/s	78/s	10,836
全 9 項目	署名+コミットメント	140/s	633/s	40/s	15,156
mid のみ	分散のみ	110,260/s	6,225,154/s	109,070/s	756
mid のみ	署名	3,794/s	11,176/s	700/s	1,204
mid のみ	署名+コミットメント	1,253/s	5,634/s	352/s	1,684

実務者の読み方

見立てが動くときに動かすのは mid だけでよい。全項目を毎回配り直す必要はなく、それだけで 9 倍速い。

技術者の読み方

外した予測: ノード側が律速だと予測したが、MM は (項目 × ノード) ごとに署名し、ノードは自分宛だけ検証するので出す側が 7 倍重い。ただしノードは全社から受けるので、社数を掛けると律速に戻る。

出所・注: maker_updates.json (host-a、n=400)。全アームで復元と受理を検証済み。帯域は最重量でも 1 更新 15KB で、制約にならない。

MMは値付けをリアルタイムで更新できるか (2/2 意味)

現実の MM が必要とする更新頻度

FX のストリーミング	1-10/秒
社債の RFQ 自動値付け	毎秒 1 回 - 毎分 1 回
上場オプション (サーフェス)	毎秒数回
HFT の株式	毎秒数千回

700/秒は上 3 つより 2-3 桁上。

届かないのは HFT の株式だけで、それは「水準を当てる」商売。

この設計が対象にするのは「在庫と即時性を売る」商売である。

ノード側は 1 コアの数字で、シェアごとに独立なので並列化する。host-a は 64 vCPU。5 コア割けば MM 側が律速に戻り 3,794/秒。MM 側は並列化できない (1 社 = 1 ディーラー)。

つまり東京の MM が、24 秒かかるクロスリージョン委員会に対して毎秒 700 回見立てを更新できる。§5 の広域配置の話と直交する。

外した予測の一覧

何を予測したか	予測	実測	向き
役割分離による通信量の増加	+5~25%	+0.3%	過大
出力マスクによるラウンド増	変わらない	-6 ラウンド	逆
EVM 上のスカラー倍のガス	7 万~20 万	302,401	過小
範囲証明の分割	+40% / +60%	+19% / +3%	過大
advanced composition の効き	改善する	この条件では悪化	逆
大陸間の 1 クォート	約 18 秒	23.0 秒	過小
use_ref の通信量	+1~3%	+0.26%	過大
MM 更新の律速はノード側	ノード	MM 側 (社数次第で戻る)	逆
ブロック範囲の問いは公開約定数で説明され尽くす	$R^2 > 0.98$	0.95~0.97、検定自体が誤り	過大

9 件中**当たったのは方向だけで、桁が合ったものは少ない**。これは「予測してから測る」を続ける理由であって、やめる理由ではない。予測を書かずに測ると、出た数字を後から正当化してしまう。

最後の 1 件は**外し方が違う** — 数字ではなく、 R^2 を見ることで検定自体が誤った検定だった。両方とも活動量で上がるので、高い R^2 は最初から自明だった。

§6

証明と決済

中身を見ずに、正しさだけを確認める

何が証明でき、何が証明できないか

プロトコルが出す証跡	効く先	効かないもの
スロット受領証（署名、時刻・市場・期限入り） 見積もり正当性証明	執行時点の記録、ノードの不作為・二重署名の立証 最良執行方針の履行証明（返した価格が本当に最小）	誰が発注したかの本人確認 価格以外の執行品質（速度・確実性）
値付けルールの監査 在庫状態の連鎖証明 決済の状態根と保存則 選択的開示	MM の社内統制の証跡 二重帳簿・遡及改竄の検出 受渡の完了、二重決済の防止 監査人・当局への個別取引の開示	統制が機能していること自体 在庫の存在（裏付け資産） 正当な発行者だけが発行したこと 網羅的なモニタリング（設計上できない）

実務者の読み方

右列が重要。「監査可能」は「統制がある」ではない。証跡は材料であって統制ではない。

技術者の読み方

「X を証明した」と「X が現実と対応する」は別。暗号が閉じられるのは前者だけで、後者は必ず外側の仮定になる。

決済 — zkPI と DeFMI

zkPI 中身を見せずに支払いを指図する

金額・価格・銘柄を開かないまま、会場の決定が正当だったことを証明付きで運ぶ。

ヌリファイアで二重使用を止め、期限で失効させる。

DeFMI 2つのレグを一緒に動かす

決済層は保存則・非負・二重決済だけを検査する。
いくらで何をやり取りしたかは読まない。

1本のトランザクションか、アダプタ署名+期限か。

16 スワップ時	単一トランザクション	アダプタ署名
検証	282.2 ± 0.2 ms	291.8 ± 0.3 ms (+3.4%)
エクスポージャ窓	なし	最低 1 ブロック
レグの結合可能性	そのトランザクションが結合そのもの	暗号的に無関係

非結合性のコストは検証 +3.4% と 1 ブロック分のヘルシュタットリスク、それだけ。異なるチェーン間だと選択の余地がなくアダプター択で、露出を決めるのは暗号ではなく両台帳のファイナリティ（秘密の回復と署名の適応は 0.06 ms、ブロック時間は 3–5 桁大きい）。

出所・注: DEFMI.md §6.1。EVM 実装は 1 決済 60,784 gas、スカラー倍 1 回 302,401 gas。

在庫はどこにあるのか

率直に言うと、いまはオフチェーンで、MM の自己申告である。ただし書き換えられない形にはなっている。

連鎖が証明すること	内容	開けずに済むもの
算術	新在庫 = 旧在庫 - 約定分。3 つのコミットメント上の線形関係	在庫の値
範囲内 連続性	新在庫が自分で約束した上限の中にある 各手順が直前の状態を名指すので、古い状態の再生や 二重帳簿が連鎖を壊す	上限も、位置も —

上限を公開の帯にすると会場最大の MM に合わせる必要が出て他の全員に無意味になる。だから上限自体をコミットメントで隠し、以後の全手順を同じコミットメントに対して証明する — 違反した後で約束を緩めることもできない。

証明していないこと。これは MM が公表した状態が正しく遷移したことを示す。公表した状態が、計算ノードが実際に使った在庫と同じであることは示していない。

出所・注: zk/state_audit.py。実測: 1 手順の証明 約 7.9 ms、検証 約 14 ms、4,960 バイト (host-c、ed25519)。

オンチェーン在庫は可能か — 3層に分けて答える

1. 連鎖 ↔ DeFMI の残高 易しい。どちらも同じ群の Pedersen コミットメントなので、「2 つが同じ値にコミットしている」ことはシグマ証明 1 本で示せる。DeFMI 側は既に残高をコミットメントとして持っている。つまり「オンチェーン在庫」は原理的に不可能ではない。
2. 連鎖 ↔ MPC が実際に使った値 ここが本当の壁。コミットメントは ed25519/ristretto の群、MPC は自前の素体 (約 2^{127}) で動く。群が違っているので、コミットメントの開示を回路の中で検査できない。これは値付けルール側にも同じ形で開いている、たった 1 つの穴であって、別物ではない。
3. 在庫と現実の保有 暗号の外。オフチェーン資産の裏付けは、証明ではなく保管と監査の問題。

実務者の読み方

在庫は業者の手の内そのもの。オンチェーンにすると「数字を公開する」ことではなく「コミットメントを台帳に置いて突合可能にする」こと。読めるようにしたら商売にならない。

技術者の読み方

コストの目安は既に測ってある。コミットメント付きで配る経路が全項目 40/秒・見立てのみ 352/秒 (16 社時) なので、必要な更新頻度には十分足りる。

結論: (1) は実装すればできる。(2) はこの研究に残っている最大の未解決点で、値付けと在庫で 2 つではなく 1 つ。(3) は制度の側。

その壁を壊せるか — MPC を同じ群の上で走らせる

Shamir の復元は線形なので、各ノードが g^{v_i} を公開して Lagrange 係数で合成すると $\prod_i (g^{v_i})^{\lambda_i} = g^v$ となり、 v を一切開かずに g^v が出る。これをコミットメントと突き合わせれば束縛が成立する — 指数の演算と MPC の演算が同じ体のときだけ。

	ラウンド	全体通信	実時間@15ms	実時間@120ms
MP-SPDZ 既定体	64	19.4 MB	3.621 s	27.603 s
ed25519 スカラー体 (253 ビット)	64	38.7 MB	3.877 s	29.179 s
比	1.00 倍	2.00 倍	1.07 倍	1.06 倍

ラウンドは動かない。通信はちょうど 2.00 倍 — 体要素が 16B → 32B になった分だけ。実時間は 15ms で 7%、120ms で 6% (クロスリージョンの方が安い。ラウンドが増えず、そこでは往復が支配的だから)。つまり公開検証可能な見積もり証明が 6~8%で買える。

実務者の読み方

「本当に最小だった」を誰でも後から検証できる状態が実時間+7%で手に入る。看板の主張を裏づける部分なので払う価値がある。

技術者の読み方

当初は 2.02 倍 / 14.3 倍 と報告していた。原因は素数をコンパイル時に埋めたことで、MP-SPDZ は毎回それを警告していた。体幅は -F、素数は実行時が正しい経路。

出所・注: matched_field.json。全アーム平文照合済み。7 倍違う数値を 4 つの判断の前提に使っていた。

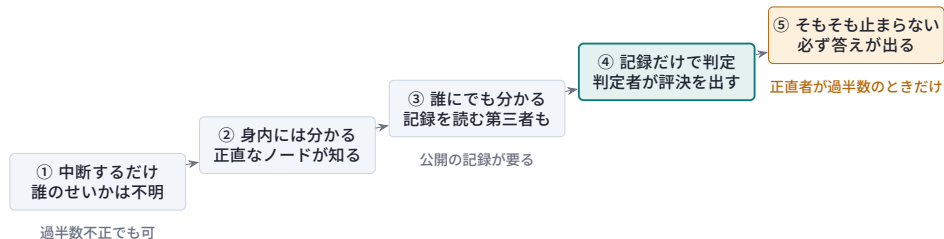
§7

不正が起きたら誰が分かるか

「止まる」「誰かが分かる」「止まらない」は別のこと

不正が起きたとき何が分かるか — 5通りある

7 ノードの 1 つが手順から外れたとする。残りの世界に何が分かるか。



実務者の読み方

「値段は正しい」「誰が壊したか」「そもそも壊れない」は別々に買う。

技術者の読み方

公開検証可能性はこの並びに入っていない（直交軸）。公開検証可能でありながら①で止まることはあり得る。

出所・注: IOZ (CRYPTO 2014)、Küsters ほか (CCS 2010)、GSZ (CRYPTO 2020)。ACCOUNTABILITY.md。

本スタックは3つの機構が3つの別の場所にいた

配られたシェアへの署名 (roles.py)

④

ただし範囲は 1 メッセージだけ

ノードがエンジンに実際に入れる値

なし

BINDING.md が丸ごとこの穴の話

MPC 本体 (malicious-shamir)

①

MP-SPDZ の README: 「手順から外れたことが検知される」

実測で確かめた。ノード 3 の前処理を 1 バイト壊しても、ノード 1 を壊しても、出る文字列は完全に同じ (Fatal error at fp2000-0:3 ... inconsistent Shamir secret sharing。あの 3 は命令位置でノード番号ではない)。さらにノード 5・6 のときは他のノードが先に接続断で落ちるので、「誰との回線が切れたか」という自然な推測は別のノードを指す。

出所・注: 4 ノード分の実測は artifacts/locate.json。

シェアは元々ただの符号語だった — 復号すれば名前が出る

Shamir のシェアは **Reed–Solomon** 符号語である。
 $n = 7$ 、次数 $t = 2$ なら $RS[7, 3]$ 、最小距離 5、訂正できる誤りは 2 個。

そして $T = 2$ 。この配備が耐えると宣言している不正の数と、名指しできる数がちょうど一致する。

エンジンは訂正できるデータに対して**検出だけ**していた。

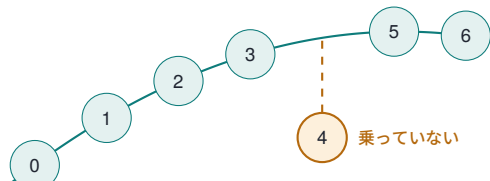
パッチ適用後 (host-a、実測)

ノード 0/1/3/5/6 を壊す → そのノードを名指す

{1,4} を壊す → 両方を名指す

{0,2,5} を壊す → 能力超過と言って拒否

次数 2 の多項式 — 正直な 6 点はこの上に乗る



Berlekamp–Welch は正しい値と誤り位置多項式を同時に返す。根がそのままノード名。局所計算、**追加ラウンド**はゼロ。

費用は通信。開示は 7 個中 5 個しか集めていなかったのも、 $T = 2$ を名指すには全部要る。

本計算 **8.000** → **12.000** (1.50 倍)、前処理込み **48.235** → **64.236** (1.33 倍)、**ラウンドは不変**。

出所・注: パッチは `rust/qomm-mpc/patches/locate-inconsistent-shares.patch`、実測は `artifacts/decode_patch.json`。

もう半分 — 「入力を偽ったノード」は復号では名指せない

加法的シェアを差し替える

別の値の正当なシェアになる

矛盾がない → 復号
すべき誤りが無い

ここで効いたのは、配る側が既にシェアごとのコミットメントを公開していたこと（`check_share` と `adds_up` のために元からある）。足りなかったのは開示の切り方だけだった。

	開く命題	失敗が言うこと
従来 ノードごと	$s = \sum_j c_j v_j + m$ を $\sum_j c_j C_j + C_m$ と照合 $s_p = \sum_j c_j x_{p,j} + m_p$ を $\sum_j c_j C_{p,j} + C_{m_p}$ と照合	どれかの入力 ノード p がやった

実務者の読み方

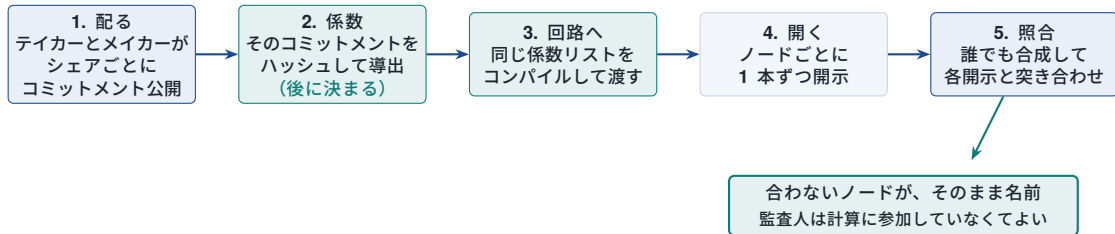
「どれかが不正」では処分できない。「ノード 4 が」なら契約も訴訟も動く。そしてその差の値段が+1 ラウンド・通信+0.39%だった。

技術者の読み方

$\sum_p s_p$ が元の開示なので厳密に強い。係数は全入力が入ってから開ける ρ のべきなので健全性は m/p — 2^{-245} 。幅の予算は不要になった。

出所・注: 実測 `artifacts/sound_check.json`、回路側 `artifacts/per_party_circuit.json`。

テイカーが送ったものと、メイカーが公開したものの同一性



ノード 4 がテイカーの数量を差し替える → ノード 4 を名指し。正直な実行では誰も名指されない。

走らせて初めて出た不具合が 2 つ。(1) 生成器は**固定の係数**を出していた — 係数を事前に読めるなら、ノードは打ち消す誤差を選ぶので検査は何も証明しない。(2) 体を間違えると**全 7 ノードを有罪にする** — 128 ビットでコンパイルしコミットメントが 253 ビット体にあると開示が巻き、完全に正直な実行で全員が名指された。設定ミスと全面侵害を運用者は区別できない。

出所・注: `scripts/run_identity.py`、`artifacts/identity.json`。

頑健性 (⑤) は費用ではなく節約だった

$T=2$ of $n=7$ は $t/n = 0.286 < 1/3$ 。GSZ は $t < n/2$ で情報理論的に⑤を達成し、 $t < n/3$ なら **broadcast** を通常の通信路で模倣できると述べる—— この配備は **broadcast** の仮定すら要らない。

1 ノードあたり体要素／乗算	要素	ラウンド
準正直 Shamir (前処理込み)	5.714	4
悪意 Shamir、本計算だけ (いま動かしているもの)	8.000	3
悪意 Shamir、前処理込み	48.235	17–23
GSZ 2020、⑤を達成、前処理という段階を持たない	5.5–7.5	—

⑤は、いま動かしているものの本計算だけより安い。前処理込みなら 6.4 倍安く、しかも中断しない分だけ強い。そして⑤は **GSZ** を書かずに手に入った—— 次ページ。

出所・注: 計測器は信じる前に照合した —— 準正直の腕が 5.714、GSZ の挙げる 5.5 と 4%差。artifacts/multiplication_cost.json。

⑤を実装した — 王を消せば復号で足りた

効く線は $t < n/3$ でなく $n \geq 4t + 1$ 。次数 $2t$ の積がそのまま訂正でき、区切りも脱落者も要らない。 $T=2$ なら $n = 9$ (7 は能力 1 で **1 っ足りない**)。

王が弱点 $2t + 1$ 点から補間して配り直すので、嘘つきの王は整合した値違いの共有を配れる。だから消す 覆われた積は秘密でないので全員に開き、各自が復号して局所に引く — 言葉を取る相手が消える。

嘘つき	答え	名指し	$n=9, T=2, \text{host-a}$
なし	正しい	—	8 ラウンド (王ありは 9)
$\{0\} / \{8\} / \{0, 1\}$	正しい	その者ら	止まらない
$\{0, 1, 2\}$	—	—	能力超過と言って拒む

実務者の読み方

18.222 要素/者/乗算 — 配備中の 64.236 に対し **1 者 0.28×**。代金は帯域でなく **2 社** 増えること。

技術者の読み方

頑健なのは乗算の次数低減だけ。二重共有・出力開示・入力相は別だが、前処理は入力を持たず中断してよい。

出所・注: robust-atlas.patch、artifacts/robust_atlas.json。予測 11.3 は **1.6 倍外した**。

テイカー側の不正 — 探りに値段をつける

テイカーは約定する気のないリクエストを投げて価格曲線を無料で読める。`is_real` のカバートラフィックはメイカーに対して隠すもので、聞く側には効かない。

従来（気配）

毎回見積もりそのものが返る

→ 1 回のリクエストで価格の全ビット

→ 費用ゼロで曲線を写せる

確定水準つき（注文）

水準 L をコミット。 $q \leq L$ なら約定

約定しない → (0, 0)（実測）

→ 約定ビット以外は何も学ばない

実務者の読み方

「気配」を「注文」に変えるだけで、新しい暗号は要らない。代償は **last look** を手放すこと。

技術者の読み方

探りは止まらない。「これより悪い」は無料。費用がかかるのは「市場が良い」と知ることだけ — それが約定だから。

当初 **+9 ラウンド**・**+1.68%**（予測 +2~8・0.5%未満 — **両方外した**）。

原因も書いてあった：トーナメントの比較は 16 並列で割り勘になるのに、約定比較だけが単独で深さを丸ごと払っていた。

単独だった比較を、最終段に相乗りさせる

$\min(a, b) \leq L$ は $a \leq L$ と $b \leq L$ で決まり、どちらの被演算子も最終段が走る前に存在している。

だから最終段は 1 組でなく 3 組を 1 層で比較し、選択も 1 回でなく 2 回を 1 層で行う。

```
best = (a<=b).if_else(a, b)
fill = (a<=b).if_else(a<=L, b<=L)
```

	ラウンド	通信量
素（拘束なし）	70	—
拘束・単独比較	79 (+9)	+1.68%
拘束・最終段に畳み込み	72 (+2)	+3.35%

実務者の読み方

9 ラウンドのうち 7 が戻った。約定拘束の遅延面の代償は、ほぼ無くなったと言ってよい。

技術者の読み方

通信量は倍になった（予測は「ほぼ不変」）。単独版は比較 2 回、畳み込み版は 3 回で、63 ビット比較 1 回が 7 ノード全体で約 **0.545 MB** — 実測 0.546 と 0.544。深さを幅で買った、このプロジェクトが組み立てられている期待の余地は

§8

先行研究のどこにいるか

取り下げた主張が 3 つ、残る差が 2 行

6つの系を2軸に置く



一文で。先行研究はすべて「回路が正しく評価された」を監査する。ここは「これが最良の価格だった」を監査し、同じ開かないコミットメントを決済まで運ぶ。

出所・注: 一次資料 6 本を読んだ結果。詳細は [POSITION.md](#)。

取り下げた主張が3つ

以前の草稿が書いていたこと

実際は誰のものか

MPC を公開監査可能にした
VOLE-in-the-Head で耐量子な監査可能 MPC
金融で稼働する最初の秘密計算

Baum–Damgård–Orlandi, SCN **2014**
Baum–Zok, eprint **2026/337** (2 月)
Prime Match, J.P. Morgan, **2023**

そして手続き上の穴。読んだ一次資料 6 本のうち **4 本は他の論文の関連研究節から出てきた**。自前の検索では出てこなかった。辿るのは効くが、それは最初の検索が弱いという意味でもある。

実務者の読み方

機構は全部既存。Pedersen、Bulletproofs、Groth–Kohlweiss、nullifier、FROST、VOLEitH。それぞれに論文がある。新しいのは繋ぎ方と、測った値段。

技術者の読み方

2014 の設計図との差は、正直者過半数 ($T=2/7$)、入力当事者と計算当事者の分離、そして体を揃える代金が **2.00 倍**だと測ったこと。その代金はどの論文にも書かれていない。

監査可能性の代金 — 実測したのは2本だけ

	何を買うか	実測
Rivinius 2022 本スタック	公開検証+帰責+頑健 公開検証	本計算 11–20 倍 時計 1.07 倍・通信 2.00 倍
Goyal–Song–Zhu 2020	頑健性のみ ($t < n/3$)	5.5–7.5 要素

この差は「誰が上手に作ったか」ではない。監査を貼る場所である。Rivinius は全ワイヤの全シェアにコミットするのでコミットメント方式が乗算の中に入る。ここはメイカーのポリシーにコミットし、機構が適用されたことを後から 1 本の証明で示すので、MPC は広い体で走るだけ。

彼らは帰責と頑健性も出している。こちらは（この節までは）どちらも出していなかった。両方書かないと比較にならない。

正直者過半数をやめたらどうなるか — 答えが2つに割れる

本計算だけ



前処理込み



実務者の読み方

見積もり 1 回あたり 7 ノードで **1.43 GB**、2 ノードでも **238 MB**。専用 1Gbps で裏に回しても 11.4 秒 / 1.9 秒に 1 件が天井。いまの設計は **0.02 秒に 1 件**で、帯域には届いていない。

技術者の読み方

誰も 7 ノードではやらない。過半数不正の意義は「2 人中 1 人が正直なら足りる」なので、比べるべきは 7 ノード Shamir 対 2 ノード MASCOT — 性能でなく統治の選択。そして⑤が原理的に不可能になる。

出所・注: 1 ノードあたり体要素 / 乗算、 $n=7$ 、128 ビット体、host-a 実測。棒は行ごとに正規化してある — 8.000 と 26,894 は同じ紙に載らないので、上下の行で長さを比べてはいけな。準正直・過半数不正でも 80 倍なので、効いているのは敵の強さではなく何人まで不正でいいか。

`artifacts/dishonest_majority.json`。

前処理は「一部」ではなく「ほぼ全部」だった

ラウンドが消費する相関乱数は注文に依存しない。依存するのは回路の形だけで、形は一度コンパイルしたら動かない。

だから注文が来る前に作れる。それがいくらの価値かを測った。

プロトコル	前処理	ラウンド	送信	時間
malicious-shamir	その場で生成	62	2.738 MB	55.8 ms
malicious-shamir	ファイルから	44	0.436 MB	25.5 ms
atlas	その場で生成	100	0.614 MB	77.7 ms
atlas	ファイルから	91	0.131 MB	56.3 ms

実務者の読み方

オンラインで残るのは **party 0** のバイトの **16%**（全体では **19%**）、ラウンドの **71%**。スロット間の空き時間に前処理を回す設計に、はっきり値打ちがある。

技術者の読み方

予測を外した。 バイトの節約を「30%以上」と書いて、実測 **84%**。前処理は送信元の一つではなくほぼ全部だった。

ただし測ったのは「オンライン相の大きさ」で、前処理を作る費用ではない — 使ったのは信頼できるディーラーなので。

誰が口座を持ってよいか — 封をして名簿に載せる

口座は $A = a \cdot G$ で、誰でも勝手に作れる。許可が要るのは審査を受けたことの方である。

実務者の読み方

名簿に載せるのはアドレスではなく「封筒」
 $C = a \cdot G + r \cdot h$ 。足した瞬間に見えるのはランダムな点だけ。

アドレスを載せると、入会の時刻から誰のアドレスが分かり、その後の決済が全部たどれる。封をしないとそこが塞がる。

技術者の読み方

使うときは一対多証明で「この組のどれかが自分だ」と示す。組は固定する — 毎回選び直すと**重ね合わせて絞り込める**から。

基点の素のベキ乗であることの **Schnorr** 証明が要る。無いと $A + \delta h$ が任意の δ で通り、**1 回の審査から無限にアドレスが作れる**。

隠れ場所の大きさは公開する。ダミー席はハッシュで作るので運営ですら開けられず、件数が「主張」でなく「事実」になる。

入会があったこと自体を隠すことと、隠れ場所の大きさを検証可能にすることは、同じ情報である。どちらか一方しか取れない — 前者を隠すには本物と見分けのつかない詰め物が要り、そうすると本物の数も誰にも数えられなくなる。

DvP は必ず同じ台帳に置く

2つの脚を一緒に動かせるのは、両方が同じ台帳にあって1つの関数が決めるからである。
状態を共有しない2つの台帳には、その関数が無い。第三者なしの公正交換は一般に不可能なので、
何かが片側からもう片側へ渡らなければならない。

実務者の読み方

現金脚も決済チェーンに置く。参加者は事前に持ち込んで残高を持ち、決済は原子的に行い、後で出す。
既存の FMI とまったく同じ形である — CSD の決済口座はプリファンドするもので、為替や資金調達はその外側で起きる。

技術者の読み方

別チェーンに置くと原子性の選択肢が無くなる。アダプタ署名と期限しかなく、誰かが待ち時間のリスクを負う。

本当に必要なのは現金が決済チェーンに来られないときだけ — 日銀ネットの中央銀行マネー、発行体が展開しないステーブルコイン。そのときは `pvp.rs` の構成になる。

出所・注: DEX との接続は決済の中ではなく資金手当での経路に置く。規制対象 FMI の決済経路に AMM を入れたい人はいない。

§9

制度上の位置

どの口座を経由し、どの法律に触れるか

清算・受渡のどこに置かれるのか



社振法では振替口座簿の記録そのものが権利なので、DeFMI は権利記録にならない。二重記帳になり、ずれたときの正解は常に振替口座簿の側 — だから照合は機能ではなく、この形を取る代金である。

実務者の読み方

清算者は差し替えられる。JSCC のままでも DeCCP でも、上流と下流は変わらない。

技術者の読み方

債務引受は掛け算 — 証明が要らない。帳簿の平坦性は恒等式。

出所・注: 橙は、その段階でいま平文で見えるもの。照合 `defmi/reconcile.py` (4,096 件 30 ms)、債務引受 `defmi/ccp.py` (16 μ s/取引)。

日本の制度に置くと、要点は3つ

1. 決済層は振替口座簿を置き換えられない。社振法では振替口座簿への記録そのものが権利なので、DeFMI が別に持つ台帳は権利記録にならない。取れる形は二択 — 既存の振替制度の鏡像として動くか、金商法 2 条 3 項の電子記録移転権利のルールに乗るか。同一銘柄で両方は取れない。
2. QOMM が何の業かは、約定がどこで成立するかで決まる。価格を返すだけで約定は相対なら媒介（登録）、QOMM 自身が売買を成立させるなら私設取引システム（認可）。説明の仕方ではなく誰と誰の間で契約が成立するかで決まる。国内には ST の PTS「START」（ODX、2023 年 12 月）という先例がある。
3. 逆に、最良執行義務との相性は非常に良い。16 社の気配を同一条件で評価して最小値を返す仕組みは最良執行方針にそのまま乗り、しかもその価格が本当に最小だったことを公開検証可能な形で証明できる。通常の最良執行方針が持てない性質。

対象利用者は **KYB 済みの法人・機関投資家**。設計上そうになっている（KYB 発行者が法人を認証する）。リテールを入れると適合性・書面交付・広告規制・投資者保護基金が乗り、要件が質的に変わる—— とくに個人に鍵の保管と復旧を求めるのは投資者保護の観点で成立しにくい。

出所・注: 法律意見ではない。制度上の論点を技術者が見落とさないための地図であり、適法性は運営主体と法域ごとに弁護士の確認が要る。

日本の外を見ると条件が変わる

設計から出てくる必要条件は 3 つ — **N1**: 公表しない気配のまま評価できる場の区分、**N2**: 決済台帳が権利記録になれるか、**N3**: 分散台帳から届く強いファイナリティの資金レグ。

	N1 場の区分	N2 台帳＝権利	N3 資金レグ	既に動いている実例
日本	PTS 認可 (30 条)。重い	振替ルールでは不可／ ST レールなら可	日銀ネット、電子決済手段	START (ODX) 2023 年 12 月～
スイス	DLT 取引システム免許。取引・決済・カस्टディが 1 免許	可。債務法 973d 条以下 (2021 年 2 月)、台帳の記録が権利そのもの	SDX + SNB のホールセール CBDC	BX Digital に初免許 (2025 年 3 月)。デジタル債 6 本・CHF 7.5 億超を wCBDC で決済
EU	DLT MTF / DLT TSS (規則 2022/858)	加盟国ごと。独 eWpG は登録簿の記録が証券を構成	T2、MiCA の電子マネートークン	認可済み DLT インフラは 3 件 (2025 年 5 月末)
英国	Digital Securities Sandbox	サンドボックスの範囲内で可	最強。BoE のオムニバス口座で中央銀行マネーに届く	DSS 2024 年 9 月開設。Fnality は
シンガポール	RMO。機関投資家限定の区分がそのままホールセール	一番弱い。台帳＝権利の法律がない	MAS のステーブルコイン枠組み	ポンドで 2023 年 12 月から稼働 Project Guardian — 実際の金融機関が実際に取引

一番効く発見: 2024 年の MiFIR 見直しで、債券等の約定前の気配公表義務が中央指値板と定期オークションだけに絞られた。EU の非株式では RFQ システムは気配を約定前に公表しなくてよい — 秘匿は免除の端で黙認されているのではなく、立法者が選んだ既定値の側にある。

もう一つ: RTS 28 の執行品質報告は「ほとんど読まれず、意味のある比較ができない」という理由で廃止された。QOMM は、検証できないから廃止された開示の、検証できる版を作っている。

§10

限界

何を主張してよく、何を主張してはいけないか

主張してよいこと／いけないこと

言ってよい

- 注文を MM にも計算ノードにも渡さずに価格を出せる（実測済み）
- 返した価格が登録済みルールの下で最小だったことを公開検証可能に証明できる
- 不作為・二重署名・古い状態の使い回しを検出し、責任者を特定できる
- 壊れたシェアの送り主をエンジンが名指せる（1.33 倍、ラウンド不変）
- 乗算中に嘘をついた 2 台まで訂正して名指し、止まらずに正しい答えを出す（ $n=9$ ）
- 入力を偽ったノードを、配る側の公開コミットメントから誰でも名指せる（+1 ラウンド・通信+0.39%、健全性 2^{-245} ）
- 決済層は金額・価格・銘柄を読まずに保存則・非負・二重決済を検査できる
- 監査人や当局への選択的開示のフックを持てる

言ってはいけない

- 「この方式は法令に適合している」—— 適法性は運営主体と法域で決まる
- 「秘密計算だからリテールでも安全」—— 投資者保護は別の層
- 「DeFMI が振替口座簿を置き換える」—— 社振法の下では置き換わらない
- 「CCP が不要になる」—— 二者間の同時交換は債務引受を再現していない
- 「監査可能だから内部統制がある」—— 証跡は材料で統制ではない
- 「MM は注文があったことすら知らない」—— 中身は漏れないが、発生は漏れうる
- 「探りを防げる」—— 防いでいない。値段をつけただけ
- 「レート制限で足りる」—— 許可制テイカーが前提。無許可なら下限がない
- 「新規 MM に参考統計を出せる」—— 取り下げた。売れるのは集中度で、活動量は元から無料
- 「どんな不正でも止まらない」—— 止まらないのは乗算・送金・決済・...

開示は市場データではなく監査だった

当初の目的は新規メーカー向けの参考情報だった。測ったら、その用途が消えた。

必要とされた統計	誰が持っているか	結論
per-fill markout 典型的な数量 勝ち半値幅	自分の約定＋公開参照テープ — 会場だけ	会場は何も供給しない 値付けでなく資本計画の入力 唯一の穴。 だが合わせにいつてはいけない

払える値幅は自分の逆選択と在庫で決まり、競合の水準では決まらない。払えない値幅に合わせにいくのが新規メーカーの死に方で、真の値を全員に渡したときの $-334/\text{約定}$ がそれである。

メーカーに本当に欠けているのは分母である — 注文が届かないので、勝った数は分かるが何件に対して評価されたかを知りようがない。そしてその半分は最初から公開されている：決済は $1 \text{ 約定} = 1 \text{ オンチェーン命令}$ (18,197,763~65,694,220 ガス) なので、ノード 1 つで正確に数えられる。秘密なのは約定しなかった側だけ。

売り物は活動量ではなく集中度

問いはブロック範囲 $[a, b]$ で見積もりを求めた相異なるエンティティは何者か。件数でなくエンティティ数なので、範囲が何窓に及んでも 1 社の寄与は 0 か 1 — 感度が全幅で 1。件数版は $\text{cap} \times \text{窓数}$ で、幅 40 窓なら 120 対 1。

範囲幅	残差	雑音	比
100 ブロック (0.3h)	1.69	1.36	1.2
600 ブロック (2h)	6.36	1.36	4.7
3,000 ブロック (10h)	19.30	1.36	14.2
7,200 ブロック (24h)	34.25	1.36	25.2

UniswapX の実スワッパーアドレス **150000** 約定・**48000** 社、**59.6%** が 1 回だけ。公開約定数が説明するのは **0.95~0.97** で、 ϵ が買っているのは残りの部分。

残差は**フローの集中度**である。同じ 500 約定でも **500** 人から来たのと **20** 人から来たのでは別の会場で、チェーンは区別できない — 逆選択を値付けするメイカーが欲しいのは正にその区別。

閉じられない限界：測った母集団は決済済みリクエスト。分母には決済しなかった分が入り、どのチェーンにも残らない。

探しへの対策は、設計が守ろうとしている紐付けを要求する

漏れているのはプロトコルではなく商品である。両側気配は定義上 $m + \text{skew} \pm \text{half}$ で、その中点は **half** が消えて $m + \text{skew}$ 、 m は公開。つまり答えそのものが在庫を運ぶ。秘匿設計と素の設計が同点なのはこのためで、注文を隠しても届かない。

効かない対策	なぜ
内容で弾く タイミングで弾く ウォレット単位の上限	探しは整形形式のリクエスト。区別する材料がない 急がない攻撃者が破る 新しいウォレットで破る
法人単位の上限	これだけが残る。既定の 60 件/エポックは 24 (多数派が有意) と 96 (ほぼ全部) の間

その上限には「1 法人の全ウォレットを束ねる nullifier」が要る — テイカー側でそれを防ぐために会場ごとハンドルを導出しているのに。両立する構成は 1 つだけで、発行体の資格証明をゼロ知識で示し、そのエポック nullifier を出す (名前を知らずに法人単位で絞る)。

無許可のテイカー集合には下限が存在しない — 上限が効くより速く識別子を作れる。許可制テイカーはこの設計の要件である (メイカー側は参加時に分かるので不要)。

残っている宿題

項目	何が足りないか	難度
選択的開示	利用者の秘密鍵なしに個別取引を当局へ開く経路	最大。法域と無関係に実務で必須
コミットメントと MPC の橋渡し	通信 2.00 倍・ラウンド不変で閉じられる。体を間違えると監査が全ノードを誤って有罪にするので、揃える理由は 2 つある	解決済み。残るのは耐量子側
耐量子コミットメント	VOLE-in-the-Head を実装して実測した — 素体では証明の 88% が整合補正で Pedersen の 8.4 倍。そして 1 回しか開けない	開いた問い。多数回開ける RO のみの構成は誰も持っていない
頑健性（止まらないこと）	$T=2/7$ は $t < n/3$ なので取れる。GSZ の代金はいまの本計算より安い。エンジンに実装が無いだけ	最も安い宿題
MPC 内での差分プライバシー雑音生成	手順は書いて値段も付いたが、公開経路はまだ中央の標本器のまま。 $n=7, T=2$ で 28 ラウンド・63.9 MB、雑音無しの公開が 2 ラウンド・0.0022 MB なので公開の 92.9% が雑音。ただしこの手順は支持を打ち切って裾を端点に疊むので、そのまま差し替えると純粹 ϵ -DP でなくなる	値段は分かった。差し替えは未了
7 拠点の実運用	実機は最大 4 拠点まで	中（運用の話）
利用者の鍵管理	HSM・複数承認・失効の運用設計	中（技術より体制）

暗号側でいちばん大きい宿題は「速くすること」ではない。測ってみると速度はノードの置き場所でほぼ決まり、暗号的改良の余地はその 6 分の 1 以下だった。残っているのは証明したものと現実を結ぶ接着面 — 選択的開示と、群をまたぐ束縛である。

まとめ

1. 問題は構造的である。RFQ では「聞く」が「教える」なので、断った相手がお互いなしに情報を得る。
2. 注文を配り、ルールを預け、価格だけ返す。断った 15 社は何も持ち帰らない。返るのは価格と、それが最小だったという証明。
3. 何が書けるかを 1 枚の宣言で固定した。実務者は自分の建て方が書けるかを読み、技術者は何が回路になるかを読む。どの項も「使わない」を選ぶ。
4. 参照価格は順位に触れない。だから計算に時間がかかっても勝者は正しく、価格は足し算 1 回で直る — 混在させない限り。
5. 速度はノードの置き場所でほぼ決まる。そして 24 秒のドリフトは、市場がもともと持っている価格の不確実性と同じ大きさだった。
6. 残っているのは暗号の速さではなく、証明と現実の接着面である。

全ての数値に成果物（JSON）とそれを生成するコマンドが対応する。予測を外した 9 件も残してある。

公開しているもの

github.com/shukob/qomm

本体。回路生成器・MP-SPDZ 連携・監査

github.com/shukob/zkpi

証明の側。コミットメント、入力整合検査、VOLE-in-the-Head

github.com/shukob/defmi

決済の側。中身を開かない受け渡し (DvP) と EVM 実装

3 つに分けてあるのは依存の向きがそうになっているからで、**zkpi** ← **defmi** ← **qomm** の順に積み上がる。

いずれも英語のみ。ホスト名は `host-a`~`host-d` に置き換えてある。

本稿の数値はすべて `artifacts/*.json` に対応する。予測を先に `*_prediction.json` として置き、測ってから照合しているので、**外した 9 件も同じ場所に残っている** — 何を外したかが読めない測定は、当たった分も信用できない。